Northwind 2.0 Developer Edition: Employees

Ala Arman

▶ *Applies To*

Select any topic below to learn about **Employees** in the Northwind Developer Edition.

Overview

Employees has one split form which works as a list and detail form.

Using Employees

Use Employees to add new employees and update information about existing employees.

Select the **Employees** tab on the Ribbon to display employees in the Employee List form, **frmEmployeeList,** is which is a Split Form with the following features:

- The top part of the form is a single view of one employee.
- The lower part of the form is a Datasheet view of all employees.
- Selecting an employee in either part also sets focus to that employee in the other part of the split form.
- You can add, edit or delete Employees in either part of the Split Form.
- Using a split form with the datasheet view part assumes the form's recordsource has multiple records.

Potential performance impact of a Split Form

Most developers prefer to *not* load an unfiltered table or query as a recordsource for a form. Filtering a form to a single record minimizes the amount of data to be transferred from the table to the form. With a local Access table, the diffe ba
recordsource and a small unfiltered recordsource mi ti

Was this information helpful? Yes No

performance is lost, however, when the Access tables are replaced with a remote, server-based database such as SQL Server.

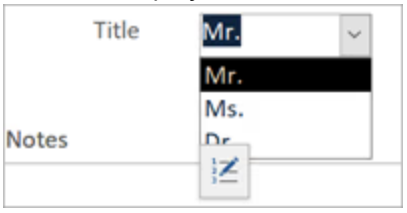
The employee table in Northwind shouldn't grow very large, so it's a candidate for an unfiltered recordsource. However, forms bound to other tables, such as order details--which potentially will grow to many thousands of records--are not candidates for an unfiltered recordsource.

EDITING EMPLOYEES

When adding or editing records in a split form, users can enter values in either in the form view or datasheet view. Some fields are required, some are optional. The form validates the presence of values in required fields. See **Required Field Validation** below.

Lookup Tables, Value Lists and Dropdown Controls

- The **Titles** dropdown limits entries to a list of pre-selected titles. Titles are in a *Lookup* table called **Titles**.
- Some expected choices are already stored in the lookup table, but users can add new titles to the lookup table, using default Access behavior for the combobox, or dropdown controls.
  - The **Limit to List** property of the **Title** combo box is set to **Yes** and the edit form for **Titles** is identified in the **List Items Edit Form** property. Selecting the dropdown on the form displays the edit icon on the bottom edge.



Clicking on the editing icon opens the **Titles** edit form, **frmEmployeeTitles**, wherein you can change or add to the list of approved titles.

NOTE: In most production environments, editing or adding titles would be limited to users with enhanced privileges.

Self-Referential Lookup fields

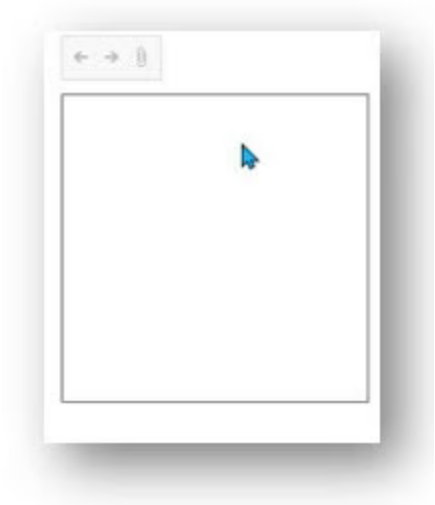
The employee function illustrates the concept of a Self-Referential ID field. To support designation of an employee's supervisor, the table has a **SupervisorID** field with **EmployeeID**s for other employees. To designate a supervisor, the **EmployeeID** for another employee is entered in this field. Access enforces Referential Integrity on this relationship.

Was this information helpful?

The **SupervisorID** is implemented in the employee form as a dropdown, or combo box; its rowsource is a filtered set of records from the Employee table. Because it is self-referential, the query must exclude the selected employee's own **EmployeeID**. For example, the rowsource for **SupervisorID** for Employee *Karen Finster* does not include Karen's own **EmployeeID**. She can't be her own supervisor.

Attachment Fields and Controls

Employee pictures are stored in an attachment field in the table. You add new images, or change existing images, using the default Access behavior for attachment fields.



NOTE: We included an attachment field in the **Employee** table to illustrate the feature in Access. Embedded images increase the size of **accdbs** and are, therefore, not considered a best practice. Carefully evaluate your environment prior to implementing it in a production application. The preferred alternative is a network folder storing all images, and a link in a text field in the table containing the path to the *image*, rather than the Attachment *field*.

Subform with Related Information

The Orders Subform (on the right side of the Employee form) displays recent orders (if any) handled by the employee. The subform lists orders in Datasheet View, sorted newest to oldest. To edit an existing order for that employee, click the hyperlinked Order ID # in the Orders subform.

Employee Form's Recordsource

A query called **qryEmployees** returns the records in the form. Using a query, rather than the table, is usually considered a best practice. The query returns records from one table only. In addition, a query can be parameterized to limit the number of records returned.

- The **Select** clause of this query uses the wildcard '*****' to return all fields from the Employee table.
- Two calculated fields in the query return **FirstName LastName** and **LastName, FirstName**.

Was this information helpful?

Calculated Values in Controls

NOTE: The calculated fields are *not* stored in the table. However, they *are* available in the form's **recordsource**.

Most controls on the Employee form are *Enabled* so you can select them with the mouse or tab into them and are *Not Locked* so they are editable.

Programming Employees

In addition to built-in Access behavior, VBA Subs and Functions in forms, in controls on forms, and in stand-alone modules perform the basic logic required for employees. The following section describes functions and the code that implements those functions.

Built-in Control Action

The form's Record Selector default behavior:

- *Left-Clicking* on the record selector to the left of a form in single view saves the current record with any edits. Saving by left-clicking the record selector triggers appropriate VBA subs and functions.
- *Right-Clicking* on the record selector to the left of a form in single view pops up a menu to perform actions like Cut, Copy or Paste a record. Deleting or cutting a record triggers the **EmployeeCanBeDeleted** function described below.
- Record Selectors must be enabled for a form for this default behavior to be available.

Referential Integrity for records in the form

Referential Integrity prevents deleting employees who have **Child** records in related tables. Therefore, Access raises an error if a user tries to delete a record with child records. The Northwind Employee form Access replaces generic, default error messages regarding referential integrity with custom messages.

Code in the form's **Delete** procedure responds to attempts either to **Cut a Record** from the right-click menu or to delete a record using the **Delete** key.

A Private Function, **EmployeeCanBeDeleted**, checks for related records for that employee in the **Orders**, **Purchase Orders**, **Employees**, and **Employee Privileges** tables.

If they are found, this function suppresses the default error message and informs the user of the reason the record can't be deleted using the same generic dialog as the **Customer** form uses to notify the user of the reason the **Customer** can't be deleted.

Validation

Was this information helpful?

Employees implements validation and standard error handling. There are two types of validation for controls.

- Required fields
- Standard formats

Required Field Validation

Three fields are required for all employees:

- First Name
- Last Name
- Job Title

In this Developer version, the form's *Before Update* event validates required fields. If a user tries to save an employee record without values for one or more required fields that don't have a value validation cancels the save and highlights any required fields that don't have a value. In the Employee form in Northwind, required field validation is handled by the *Before Update* event of the form, not by the individual controls.

The Employee form's *Before Update* event validates the presence of values for the three required fields. Saving a new or edited employee record fires the form's *Before Update* event, which calls Public Functions that check for, and respond to, the presence or absence of the required values.

The **modValidation** module contains these functions:

- **ValidateForm**
- **IsValidForm**
- **HighlightInvalidControls**
- **HighlightControl**
- **ValidateForm_RemoveHighlights**

Managing new employees

Clicking the **Add Employee** button runs the Private Sub **btnNewEmployee_Click** in order to:

- save the current record and

Was this information helpful?

- set the form's focus to a new record

'

Wildcard vs Named Fields in a Select query

The **SELECT** clause in **qryEmployees** uses the wildcard character "*****" to select **ALL** fields in the table. The wildcard query design automatically includes any newly added fields from an underlying table, which is not possible with a list of specific fields. On the other hand, one might list specific fields to restrict the returned **recordset** to only those fields needed in a display only form, for example.

Appropriate Design Choices

Your designs should be based on the appropriate approach for your requirements. Although, when feasible, preference would normally go to the approach least likely to require future maintenance (i.e., the wildcard approach).

ComboBoxes bound to Lookup Tables — Two Approaches

Related items, such as an employee's Supervisor, are displayed in Dropdowns or Comboboxes on forms. Only the Foreign Key for **SupervisorID** is needed in the form's **recordsource** query because that field is bound to the Foreign Key field (**SupervisorID**). The combobox also displays the corresponding text value.

A two-column combobox with a hidden ID column and a visible Description column makes this work. In the Employee Form, the Supervisor combobox is bound to a simple two-column query. See the **RowSource** property for Supervisor.

In some cases, however, a lookup table has no separate Primary Key, and therefore, the text value itself is the Primary Key.

When a list of possible values is small and highly stable, such as an employee's title, it is often called a Closed Domain. Changing or adding values for Title is not common. Closed Domain lookup tables are candidates for the one-column query approach.

In the Employee form, the Title combobox is bound to a one-column query of Titles. See the **RowSource** property for Titles.

Learn more

- Learn more about Split Forms
- Learn more about Referential Integrity
- Northwind 2.0 Developer Edition: All topics
- Northwind 2.0 Developer Edition: Things you should know

Was this information helpful?

Need more help?

How can we help you?

Want more options?

Discover **Community**

Explore subscription benefits, browse training courses, learn how to secure your device, and more.



Microsoft 365 subscription benefits



Microsoft 365 training



Microsoft security



Accessibility center

Surface Laptop
Surface Laptop Studio 2
Copilot for organizations
Copilot for personal use
AI in Windows
Explore Microsoft products
Windows 11 apps

Account profile
Download Center
Microsoft Store support
Returns
Order tracking
Certified Refurbished
Microsoft Store Promise
Flexible Payments

Devices for education
Microsoft Teams for Education
Microsoft 365 Education
How to buy for your school
Educator training and development
Deals for students and parents
AI for education

Microsoft Security
Dynamics 365
Microsoft 365
Microsoft Power Platform
Microsoft Teams
Microsoft 365 Copilot
Small Business

Azure
Microsoft Developer
Microsoft Learn
Support for AI marketplace apps
Microsoft Tech Community
Microsoft Marketplace
Marketplace Rewards
Visual Studio

About Microsoft
Company news
Privacy at Microsoft
Investors
Diversity and inclusion
Accessibility
Sustainability

English (United States) Your Privacy Choices Consumer Health Privacy

Sitemap Contact Microsoft Privacy Terms of use Trademarks Safety & eco Recycling About our ads

© Microsoft 2026

Was this information helpful?